

23CSE202 DATABASE MANAGEMENT SYSTEMS

S3 B.Tech CSE

Department of Computer Science and Engineering

Amrita School of Computing, Amritapuri Campus

3. Database Design Document

Group No : C1

Team Members :

Roll No	Student Name
AM.SC.U4CSE25204	Anjana R
AM.SC.U4CSE25219	Gowri Menon
AM.SC.U4CSE25230	Lekshmi Nandana
AM.SC.U4CSE25232	Meenakshi N M

Project Title : BloodBank

3.1. Detailed description of the project

[clearly describe the scenario so that the entities, attributes, relationships, users and business rules are evident from the description]

Scenario Overview

BloodConnect is a centralized Blood Donation Management System designed to bridge the gap between donors, hospitals, and blood banks to save lives efficiently. Its primary goal is to maintain interconnected records of blood donors, recipients, blood banks, hospitals, blood donations, and blood requests in a single database. By transitioning away from manual record-keeping, the system eliminates duplicate data, mitigates human errors, and prevents delays in blood allocation during medical emergencies.

System Users and Access Roles

The platform serves multiple distinct user roles, primarily administrators, hospital staff, and registered blood donors. Administrators possess comprehensive oversight, allowing them to register new donors, verify donor eligibility, update the blood inventory, approve or reject hospital blood requests, and generate system-wide reports. Hospital staff utilize the system to submit requests for specific blood groups, check real-time blood availability, and continuously monitor the fulfillment status of their requests. Meanwhile, registered donors can log into their profiles to update personal information,

track their historical donation volume in litres, and check when they are next eligible to donate.

Entities and Attributes

The relational schema relies on several core entities to function. The Donor entity captures personal and health information, including Full Name, Email, Phone, Blood Group, Age, Gender, Address, Total Donations, Total Volume in litres, and the Last Donation Date. The Blood Inventory entity tracks the aggregated stock for each blood group, monitoring the Available Units, Total Volume, and the current Stock Status. The Blood Request entity is designed to manage healthcare demands, logging the originating Hospital, Patient Name, Blood Group, Units Needed, Urgency Level (Normal, Urgent, Critical), and the current Status (Pending, Approved, Dispatched, Rejected). Finally, Donation Records capture individual donation events, noting the specific units and volume contributed by a donor on a given date.

Relationships and Cardinality

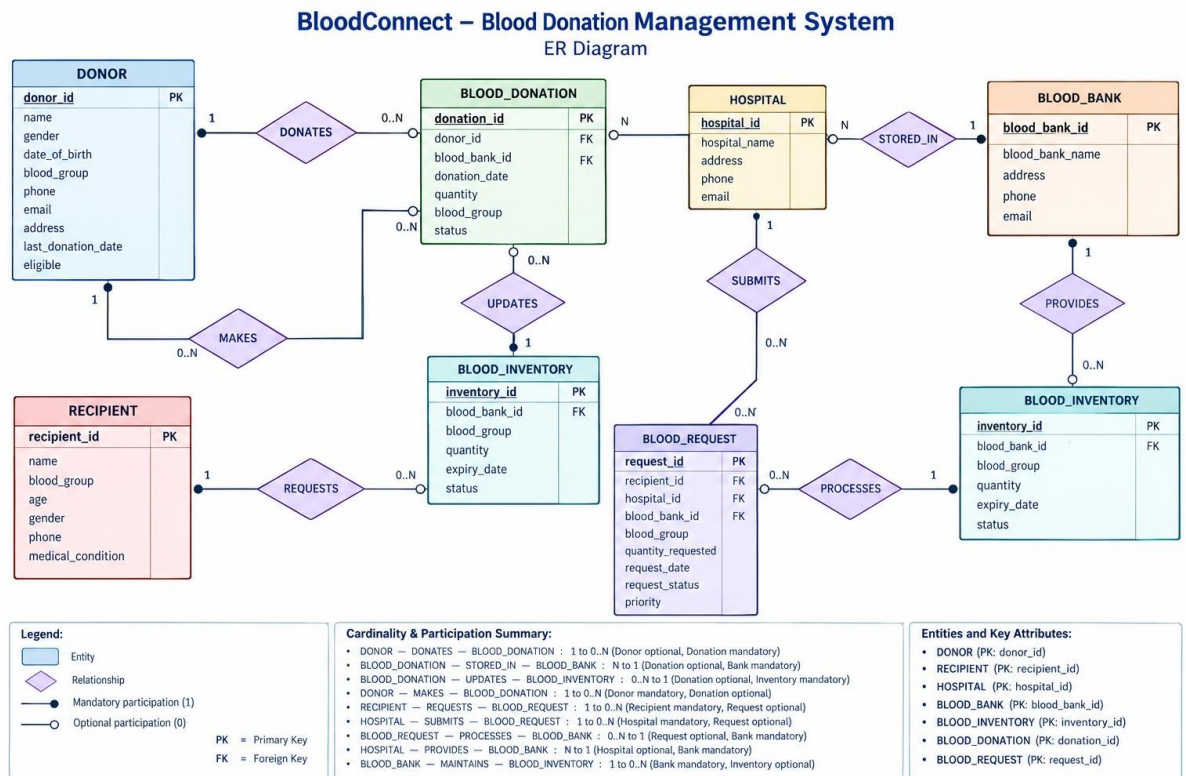
The system is built upon managing the relationships between these multiple entities. During registration, a single user account is mapped to a specific role, such as a Donor or Staff member. A one-to-many relationship exists between a donor and their donation records; a single donor can make multiple donations over time, with each transaction adding to their cumulative donation history. Similarly, a hospital can generate multiple blood requests for various patients. Each individual donation transaction has a direct relationship with the master blood inventory, as every approved donation increases the aggregated stock of that specific blood group.

Business Rules and Constraints

Several business rules ensure data integrity and system reliability. To successfully register on the platform, individuals must meet the age requirement of 18 to 65 years old. Donor eligibility is strictly validated based on the time elapsed since their previous donation date, automatically categorizing them as eligible or ineligible. The system calculates blood volume uniformly, converting donated units into litres, with standard donations measuring 0.45L. To prevent shortages, automated stock statuses act as visual triggers, flagging a blood group's inventory as "Critical" when it falls to 2 units or below, and "Low Stock" when it reaches 5 units or below. Additionally, hospital requests must specify an urgency level and cannot be fulfilled until administrators verify sufficient inventory.

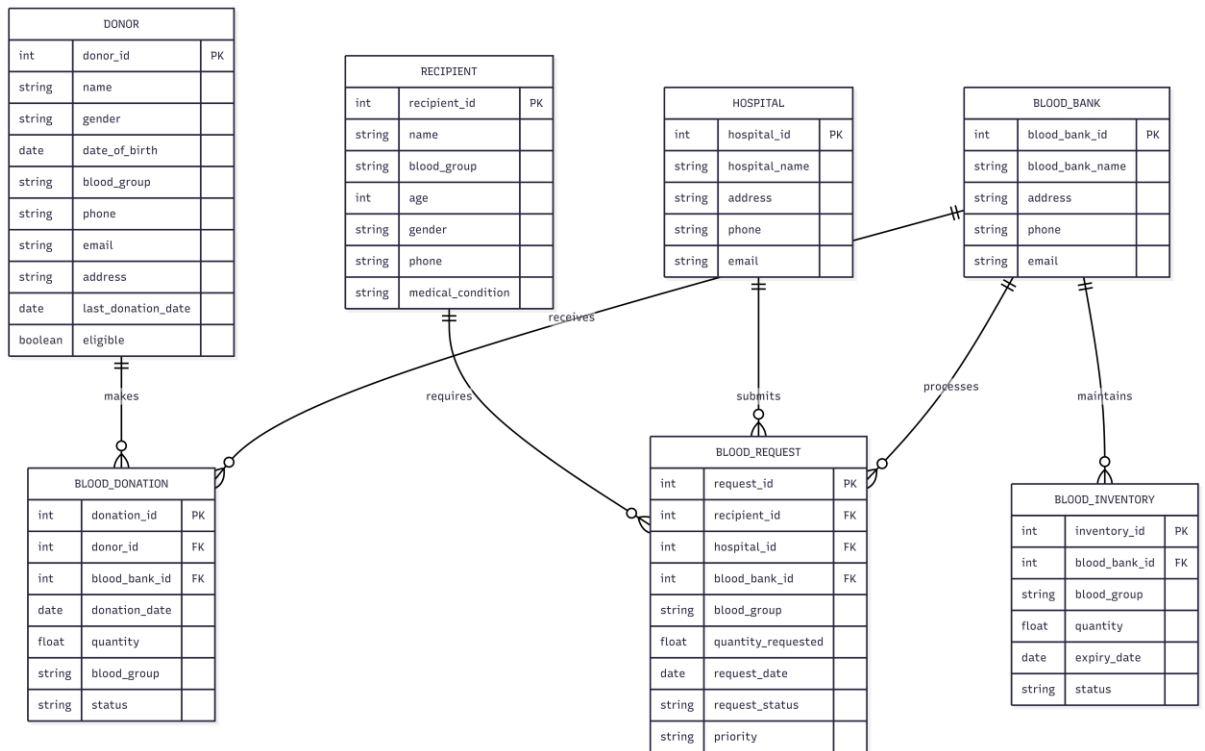
3.2. ER Diagram

[entities, attributes, relationships, keys, cardinality and participation have to be accurately represented]



3.3 Relational Schema

[Correctly converts all entities, relationships, cardinality etc to relational schema]



3.4 Normalization and Functional dependency analysis

[Correctly identifies relevant FDs and explains their significance]

Justify the selection of candidate keys with attribute closure

Demonstrate the normalization process starting from 1NF to BCNF or higher with proper justification]

The BloodConnect – Blood Donation Management System contains several interconnected entities such as donors, recipients, hospitals, blood banks, donations, blood requests, and blood inventory. Functional dependency analysis is performed to identify the relationships between attributes and to ensure that the database is free from unnecessary redundancy and update anomalies.

3.4.1 Functional Dependencies

The functional dependencies for the relations in the BloodConnect database are identified based on their primary keys.

1. DONOR

Relation:

DONOR(donor_id, name, gender, date_of_birth, blood_group, phone, email, address, last_donation_date, eligible)

The functional dependency is:

*donor_id → name, gender, date_of_birth, blood_group,
phone, email, address, last_donation_date, eligible*

Here, donor_id uniquely identifies a donor. Therefore, all the other donor attributes depend on donor_id.

2. RECIPIENT

Relation:

RECIPIENT(recipient_id, name, blood_group, age, gender, phone, medical_condition)

Functional dependency:

*recipient_id → name, blood_group, age, gender,
phone, medical_condition*

recipient_id uniquely identifies each recipient and determines all their personal and medical details.

3. HOSPITAL

Relation:

HOSPITAL(hospital_id, hospital_name, address, phone, email)

Functional dependency:

hospital_id $\rightarrow$ hospital_name, address, phone, email

Each hospital is uniquely identified by hospital_id.

4. BLOOD_BANK

Relation:

BLOOD_BANK(blood_bank_id, blood_bank_name, address, phone, email)

Functional dependency:

blood_bank_id $\rightarrow$ blood_bank_name, address, phone, email

Each blood bank is uniquely identified by blood_bank_id.

5. BLOOD_DONATION

Relation:

BLOOD_DONATION(donation_id, donor_id, blood_bank_id, donation_date, quantity, blood_group, status)

Functional dependency:

$donation_id \rightarrow donor_id, blood_bank_id, donation_date,$
 $quantity, blood_group, status$

Each donation record is uniquely identified by donation_id.

donor_id and blood_bank_id are foreign keys connecting the donation with the respective donor and blood bank.

6. BLOOD_REQUEST

Relation:

BLOOD_REQUEST(request_id, recipient_id, hospital_id, blood_bank_id,
blood_group, quantity_requested, request_date, request_status, priority)

Functional dependency:

$request_id \rightarrow recipient_id, hospital_id, blood_bank_id,$
 $blood_group, quantity_requested,$
 $request_date, request_status, priority$

Each blood request is uniquely identified by request_id.

7. BLOOD_INVENTORY

Relation:

BLOOD_INVENTORY(inventory_id, blood_bank_id, blood_group, quantity, expiry_date, status)

Functional dependency:

*inventory_id → blood_bank_id, blood_group,
quantity, expiry_date, status*

Each inventory record is uniquely identified by inventory_id.

3.4.2 Candidate Keys Using Attribute Closure

The candidate keys are verified using attribute closure.

DONOR

donor_id⁺

= {donor_id}

*donor_id → name, gender, date_of_birth, blood_group,
phone, email, address, last_donation_date, eligible*

Therefore,

donor_id⁺ =

*{donor_id, name, gender, date_of_birth, blood_group,
phone, email, address, last_donation_date, eligible}*

Since the closure contains every attribute of DONOR:

Candidate Key = donor_id

RECIPIENT

recipient_id⁺

= {recipient_id}

*recipient_id → name, blood_group, age, gender,
phone, medical_condition*

Therefore,

*recipient_id⁺ =
{recipient_id, name, blood_group, age, gender,
phone, medical_condition}*

Thus:

Candidate Key = recipient_id

HOSPITAL

hospital_id⁺

= {hospital_id}

hospital_id → hospital_name, address, phone, email

Therefore:

Candidate Key = hospital_id

BLOOD_BANK

blood_bank_id⁺

= {*blood_bank_id*}

blood_bank_id → *blood_bank_name*, *address*, *phone*, *email*

Therefore:

Candidate Key = *blood_bank_id*

BLOOD_DONATION

donation_id⁺

= {*donation_id*}

donation_id → *donor_id*, *blood_bank_id*, *donation_date*,
quantity, *blood_group*, *status*

Therefore:

Candidate Key = *donation_id*

BLOOD_REQUEST

request_id⁺

= {*request_id*}

request_id → *recipient_id*, *hospital_id*, *blood_bank_id*,

*blood_group, quantity_requested,
request_date, request_status, priority*

Therefore:

Candidate Key = request_id

BLOOD_INVENTORY

inventory_id⁺

= {inventory_id}

*inventory_id → blood_bank_id, blood_group,
quantity, expiry_date, status*

Therefore:

Candidate Key = inventory_id

Candidate Key Summary

Relation Candidate Key

DONOR donor_id

RECIPIENT recipient_id

HOSPITAL hospital_id

BLOOD_BANK blood_bank_id

BLOOD_DONATION donation_id

BLOOD_REQUEST request_id

BLOOD_INVENTORY inventory_id

All relations have a single-attribute candidate key. Therefore, none of the relations has a composite candidate key.

3.4.3 Normalization Process

The database is normalized systematically from 1NF to BCNF. The purpose of normalization is to reduce data redundancy, prevent insertion/deletion/update anomalies, and maintain data consistency.

First Normal Form (1NF)

A relation is in 1NF when:

Each attribute contains only atomic values.

There are no repeating groups.

Each row can be uniquely identified.

Each column contains values of a single type.

The relations in the BloodConnect system satisfy these conditions.

For example:

DONOR

donor_id | name | gender | blood_group | phone | email | ...

Each attribute contains a single value and there are no repeating groups.

Therefore:

All relations are in 1NF.

3.4.4 Second Normal Form (2NF)

A relation is in 2NF when:

It is already in 1NF.

No non-prime attribute is partially dependent on a part of a candidate key.

In the BloodConnect database, all candidate keys are single attributes.

For example:

donor_id → name, gender, blood_group, phone, ...

Since donor_id is a single-attribute key, partial dependency is not possible.

Similarly:

recipient_id → recipient details

hospital_id → hospital details

blood_bank_id → blood bank details

donation_id → donation details

request_id → request details

inventory_id → inventory details

Therefore, there are no partial dependencies.

Hence, all relations satisfy 2NF.

3.4.5 Third Normal Form (3NF)

A relation is in 3NF when:

It is already in 2NF.

There is no transitive dependency of a non-key attribute on the candidate key.

For example, in the DONOR relation:

donor_id $\rightarrow$ name

donor_id $\rightarrow$ gender

donor_id $\rightarrow$ date_of_birth

donor_id $\rightarrow$ blood_group

donor_id $\rightarrow$ phone

donor_id $\rightarrow$ email

donor_id $\rightarrow$ address

donor_id $\rightarrow$ last_donation_date

donor_id $\rightarrow$ eligible

All non-key attributes directly depend on donor_id.

Similarly, in BLOOD_DONATION:

donation_id $\rightarrow$ donor_id

donation_id $\rightarrow$ blood_bank_id

$donation_id \rightarrow donation_date$

$donation_id \rightarrow quantity$

$donation_id \rightarrow blood_group$

$donation_id \rightarrow status$

The donor_id and blood_bank_id attributes act as foreign keys and establish relationships with other entities. Their presence does not create a transitive dependency among non-key attributes within the relation.

The same applies to BLOOD_REQUEST and BLOOD_INVENTORY.

Therefore:

All relations satisfy 3NF.

3.4.6 Boyce-Codd Normal Form (BCNF)

A relation is in BCNF if, for every non-trivial functional dependency:

$X \rightarrow Y$

X must be a candidate key.

The relations can be checked as follows:

<i>Relation</i>	<i>Functional Dependency</i>	<i>Determinant</i>	<i>BCNF</i>
<i>DONOR</i>	$donor_id \rightarrow all\ attributes$	<i>donor_id</i>	✓
<i>RECIPIENT</i>	$recipient_id \rightarrow all\ attributes$	<i>recipient_id</i>	✓
<i>HOSPITAL</i>	$hospital_id \rightarrow all\ attributes$	<i>hospital_id</i>	✓
<i>BLOOD_BANK</i>	$blood_bank_id \rightarrow all\ attributes$	<i>blood_bank_id</i>	✓

BLOOD_DONATION donation_id → all attributes donation_id ✓

BLOOD_REQUEST request_id → all attributes request_id ✓

BLOOD_INVENTORY inventory_id → all attributes inventory_id ✓

In every relation, the determinant of the functional dependency is the candidate key.

Therefore, all the relations satisfy BCNF.

3.4.7 Final Normalized Database

The final normalized schema of the BloodConnect system is:

DONOR(

donor_id PK,

name,

gender,

date_of_birth,

blood_group,

phone,

email,

address,

last_donation_date,

eligible

)

RECIPIENT(

recipient_id PK,

name,

blood_group,

```
    age,  
    gender,  
    phone,  
    medical_condition  
)  
HOSPITAL(  
    hospital_id PK,  
    hospital_name,  
    address,  
    phone,  
    email  
)  
BLOOD_BANK(  
    blood_bank_id PK,  
    blood_bank_name,  
    address,  
    phone,  
    email  
)  
BLOOD_DONATION(  
    donation_id PK,  
    donor_id FK,  
    blood_bank_id FK,  
    donation_date,  
    quantity,  
    blood_group,  
    status  
)  
BLOOD_REQUEST(  

```

```
    request_id PK,  
    recipient_id FK,  
    hospital_id FK,  
    blood_bank_id FK,  
    blood_group,  
    quantity_requested,  
    request_date,  
    request_status,  
    priority  
)  
BLOOD_INVENTORY(  
    inventory_id PK,  
    blood_bank_id FK,  
    blood_group,  
    quantity,  
    expiry_date,  
    status  
)
```